UDACITY / MACHINE LEARNING CAPSTONE

Animal Image Classification Using Transfer Learning with ResNet18

A controlled comparison of frozen-feature learning and partial fine-tuning

Research question. Does unfreezing the final residual block of an ImageNet-pretrained ResNet18 improve animal-image classification compared with training a replacement classifier head alone?

Dataset. Balanced CIFAR-10 subset of 2,700 cat, dog and horse images: 1,800 training, 450 validation and 450 held-out testing examples.

Summary finding. Fine-tuning improved held-out accuracy from 86.00% to 90.89% and macro-F1 from 0.8590 to 0.9090. The paired accuracy difference was +4.89 percentage points (95% paired bootstrap interval: +1.78 to +8.00 points).

EXPERIMENT RECORD | SEED 42 | 3 EPOCHS PER MODEL | APPLE MPS

DEFINITION

1. Project overview and problem definition

Background. Image categorization is a practical computer-vision problem: software must infer the subject of a photograph from pixel patterns, even when pose, lighting and background vary. A useful small-scale example is distinguishing cats, dogs and horses, a task relevant to photo organization and a simplified prototype for visual tagging. The application here is educational rather than a production animal-identification service.

Motivation. Training a deep image model from randomly initialized parameters generally requires substantial labeled data and computation. Transfer learning is attractive for smaller datasets because the early and intermediate visual representations can be reused from a model pretrained on a large dataset. ResNet uses residual shortcuts to facilitate optimization of deep convolutional networks [2].

Problem statement. Given one 32 x 32 RGB image from a restricted three-category CIFAR-10 subset, predict exactly one label: cat, dog or horse. The network outputs three logits, converts them to class probabilities using softmax at inference, and selects the highest-probability label. This is supervised, single-label multiclass classification.

Hypothesis and scope

The primary hypothesis is that updating ResNet18 **layer4** in addition to the final classification layer will improve held-out macro-F1 compared with updating the classification layer alone. Both strategies begin from ImageNet-1K pretrained weights, use the same data manifest and are selected by validation macro-F1. The smaller frozen-feature model is the learned benchmark.

The aim is not to estimate performance for all photographs, every animal category or deployment environments. It is to provide a measurable, reproducible experiment with a clear baseline, refinement, held-out validation and transparent limitations.

DEFINITION

2. Metrics, benchmarks and experimental controls

Primary metric: macro-F1. For each class, precision = TP/(TP + FP), recall = TP/(TP + FN), and F1 = 2 x precision x recall/(precision + recall). Macro-F1 is the arithmetic mean of the three classwise F1 scores. It gives cat, dog and horse equal importance, even when one class is harder to recognize.

Complementary measures. Accuracy is the proportion of examples correctly labeled. Cross-entropy loss is used for training and to observe convergence. Confusion matrices describe which true categories are mistaken for others. We additionally report 95% percentile bootstrap intervals for accuracy and a paired bootstrap interval for the accuracy difference.

Benchmark / model	Decision rule	Role
Non-learned reference	Always predict a single category	33.33% accuracy on this balanced test set
Experiment A	Frozen ResNet18 backbone; train 3-class head	Learned baseline
Experiment B	Train head and last residual block (layer4)	Refinement candidate

Separation of selection and evaluation

Training and validation images originate only from the official CIFAR-10 training partition. The held-out images originate only from the official CIFAR-10 test partition. Each model checkpoint is selected using its best **validation macro-F1**; test-set measurements are calculated after this selection. The saved manifest preserves the exact sampled indices and class labels.

Predeclared success criterion. The refined strategy should improve test macro-F1 over the learned baseline, ideally with a paired accuracy interval excluding zero. This combines practical effect size with uncertainty assessment. A 33.33% class-constant baseline is a mathematical reference, not a third trained experiment.

ANALYSIS

3. Dataset and class exploration

CIFAR-10 contains 60,000 RGB images of size 32 x 32 across ten categories, with 50,000 images in the official training partition and 10,000 in the official test partition [1, 5]. We select the original labels cat=3, dog=5 and horse=7. A seeded permutation picks distinct indices within each selected category, retaining the source-partition boundary.

Split	Cat	Dog	Horse	Total
Training	600	600	600	1,800
Validation	150	150	150	450
Test	150	150	150	450
Total	900	900	900	2,700

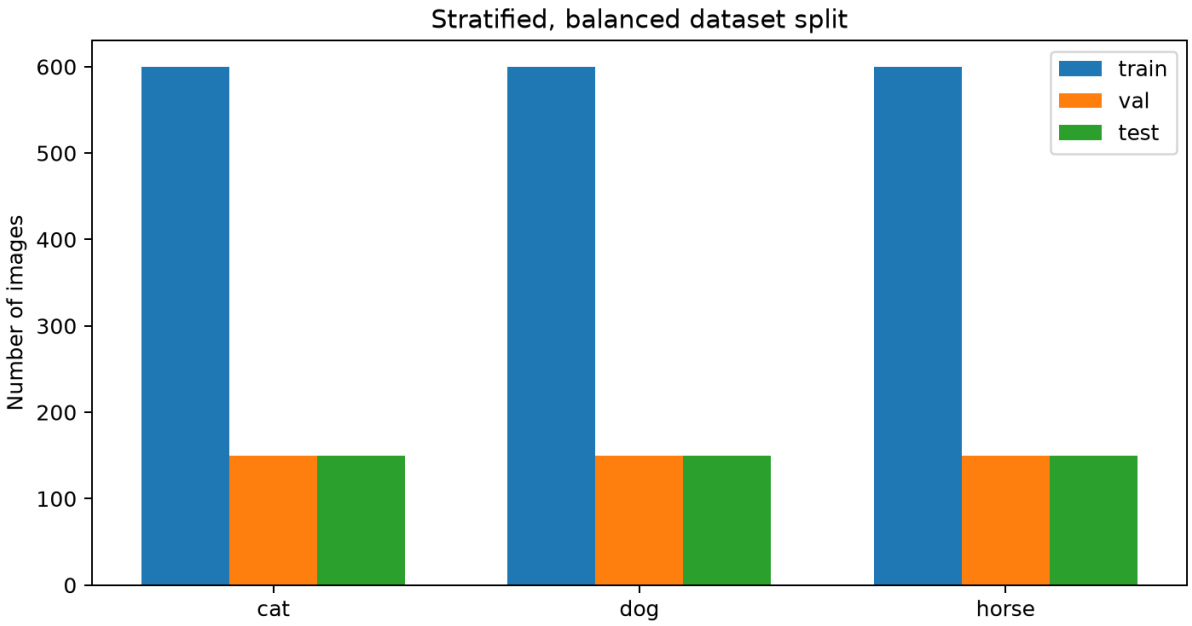


Figure 1. Every class has equal representation within each data split; the plotted counts are computed from the saved split manifest.

Data integrity. The split manifest contains an original-source identifier and integer row index for every example. The included unit test verifies the requested counts, uniqueness of source/index pairs and exclusive use of the official test partition for test examples. This prevents index reuse across local splits; it does not guarantee all photographs are semantically independent.

ANALYSIS

4. Exploratory visualization and data challenges



Figure 2. First four sampled training examples per class; originally 32 x 32 pixels, enlarged for viewing.

Observed visual characteristics. The dog row includes both a dark full-body outdoor silhouette and a tightly framed face; shape, background and image scale vary considerably even within one label. The cat examples include a dark cat against a light background and a pale cat beside a bright yellow graphic, illustrating substantial color and scene variation. Horse examples range from a close side view to an image with a rider, showing that context is not uniform.

Challenges. Enlargement makes pixelation obvious: fine textures, eyes and body boundaries are poorly resolved in many examples. Backgrounds and other contextual cues may be spuriously predictive. Cropping, overlapping animals, pose and label ambiguity could all affect predictions. Resizing to 224 x 224 makes inputs compatible with pretrained ResNet18, but cannot recover information absent in the original 32 x 32 pixels.

ANALYSIS

5. Algorithms, transfer learning and benchmark

Convolutional residual network. ResNet18 is an 18-layer residual CNN; its shortcut connections let blocks learn residual updates relative to their inputs, improving optimization compared with a plain stack of layers [2]. ImageNet pretraining supplies visual features learned across a much broader object-recognition problem. Following established transfer-learning practice [3,4], the original 1,000-class fully connected layer is replaced with a randomly initialized 3-class linear layer.

Setting	Experiment A: frozen	Experiment B: partial tune
Initialization	ImageNet1K_V1	ImageNet1K_V1
Trainable components	New fully connected head	Head + final residual block
Trainable parameters	1,539	8,395,267
Learning rate	0.0010	0.0001
Optimizer	AdamW	AdamW
Weight decay	0.0001	0.0001

Why compare these two strategies?

The frozen-feature baseline is a computationally cheap, strong alternative to learning every CNN weight from scratch: only 1,539 classifier weights/biases are updated. Allowing the last residual block to adapt gives the network substantially more trainable capacity (8,395,267 parameters), potentially improving adaptation to small CIFAR-10 animal images but increasing susceptibility to overfitting.

Important design caveat. Experiment B also uses a lower learning rate (0.0001 versus 0.001). Therefore this is a comparison of two complete training configurations, not a causal isolation of layer4 trainability alone. A future ablation should hold learning rate constant or compare a matched rate grid on validation data.

6. Preprocessing and leakage prevention

Split construction. For each target class, NumPy seeded permutation (seed=42) selects 600 official-train rows for training, the next 150 official-train rows for validation and 150 rows from the official-test partition. Records are written to `runs/splits.csv`, including split, source, original index, target and category name. The same file drives both experiments.

Stage	Applied to training	Applied to validation/test
Resize	224 x 224 pixels	224 x 224 pixels
Horizontal flip	Random, p = 0.5	Not applied
Tensor conversion	PIL RGB -> float tensor	Same
Channel normalization	ImageNet mean / std	Same
Shuffling	Training loader only	Disabled

The ImageNet normalization mean is (0.485, 0.456, 0.406) and standard deviation is (0.229, 0.224, 0.225), consistent with the pretrained model configuration [4]. Random horizontal flipping only alters training samples, not validation or test observations. No target-label information is introduced through the transforms.

Potential data and model leakage controls

The model weights are freshly loaded from the same original ImageNet checkpoint for each run; Experiment B does not begin from Experiment A's trained three-class checkpoint. The scripts load test examples only for evaluation. Checkpoint choice uses validation macro-F1, and the analysis CSVs are aligned by ordered test position and true label before any paired comparison. These controls support fair evaluation on this fixed subset.

METHODOLOGY

7. Implementation and reproducibility

The pipeline is implemented in `capstone.py` with three CLI stages: **prepare** downloads/source-checks CIFAR-10 and saves the manifest and EDA figures; **train** constructs model variants, runs training, records learning curves and saves the best validation checkpoint; **evaluate** reloads that checkpoint and writes test metrics, aligned predictions and a confusion matrix. A separate analysis script recomputes paired statistics directly from prediction CSVs.

Recorded configuration	Value
Execution backend	Apple Metal Performance Shaders (mps)
PyTorch / Torchvision	2.15.0 / 0.29.2 (reported runtime)
Seed / batch size	42 / 32
Training epochs per model	3
Loss / optimizer	Multiclass cross-entropy / AdamW
Best checkpoint rule	Maximum validation macro-F1

Implementation complications and resolution

During setup, the initially selected environment lacked PyTorch; this was resolved by activating the project virtual environment and installing dependencies. The CIFAR-10 download then failed SSL certificate verification under macOS Python. Installing `certifi` and setting `SSL_CERT_FILE` to its certificate bundle allowed the official dataset to download with certificate verification still enabled. Pretrained weights were then downloaded successfully.

All training and test evaluation were run locally; no SageMaker endpoint or ongoing AWS compute was needed. For reproduction, use the README commands, the deterministic manifest, the specified pretrained weight variant, and the saved metrics. Exact numeric replication may still vary slightly across hardware and library versions, especially GPU/MPS backends.

8. Refinement and learning-curve behavior

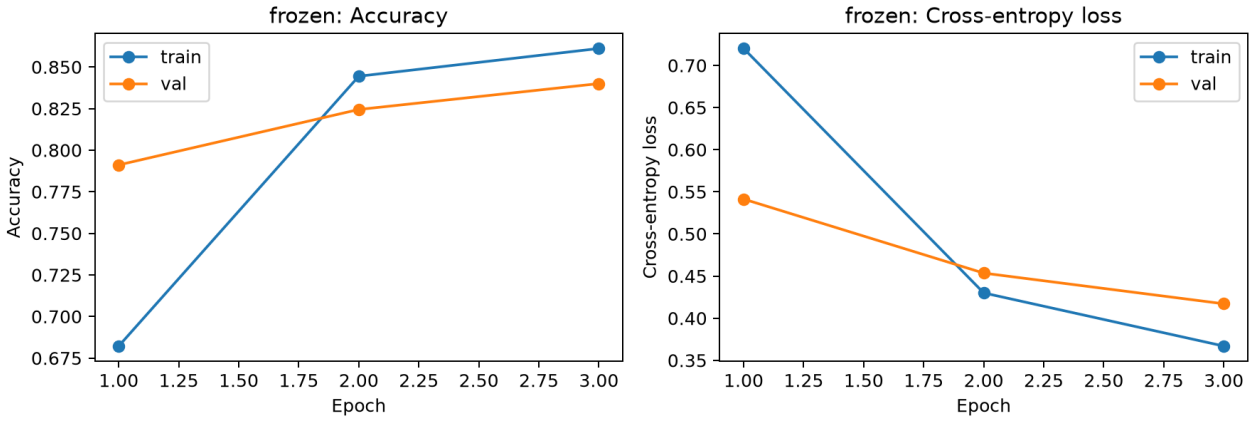


Figure 3. Frozen-feature head: training vs validation accuracy and loss over three epochs.

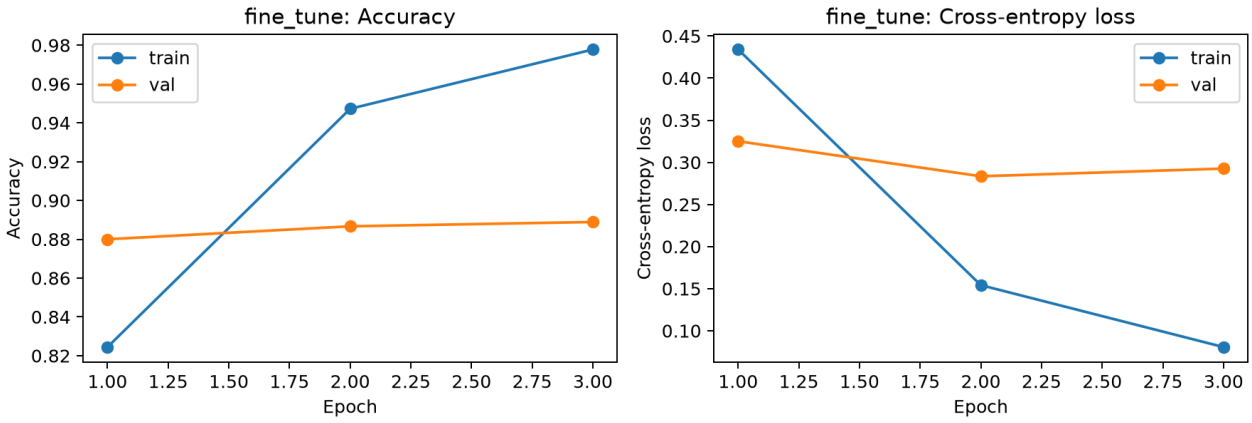


Figure 4. Fine-tuned layer4 + head: training vs validation accuracy and loss over three epochs.

Baseline convergence. Frozen-head validation accuracy grew from 79.11% to 84.00% and validation loss decreased from 0.5414 to 0.4171. Fine-tuning raised best validation accuracy to 88.89% and best validation macro-F1 to 0.8888, versus 0.8398 for the frozen model; both best checkpoints occurred at epoch 3.

Overfitting caution. In fine-tuning, training accuracy increased to 97.78% while validation accuracy stayed at 88.89%. Validation loss improved through epoch 2 (0.2835) but rose to 0.2926 at epoch 3, despite a small F1 improvement. This divergence suggests early overfitting/calibration concerns; it does not prove performance would worsen with further training.

RESULTS

9. Held-out results and principal findings

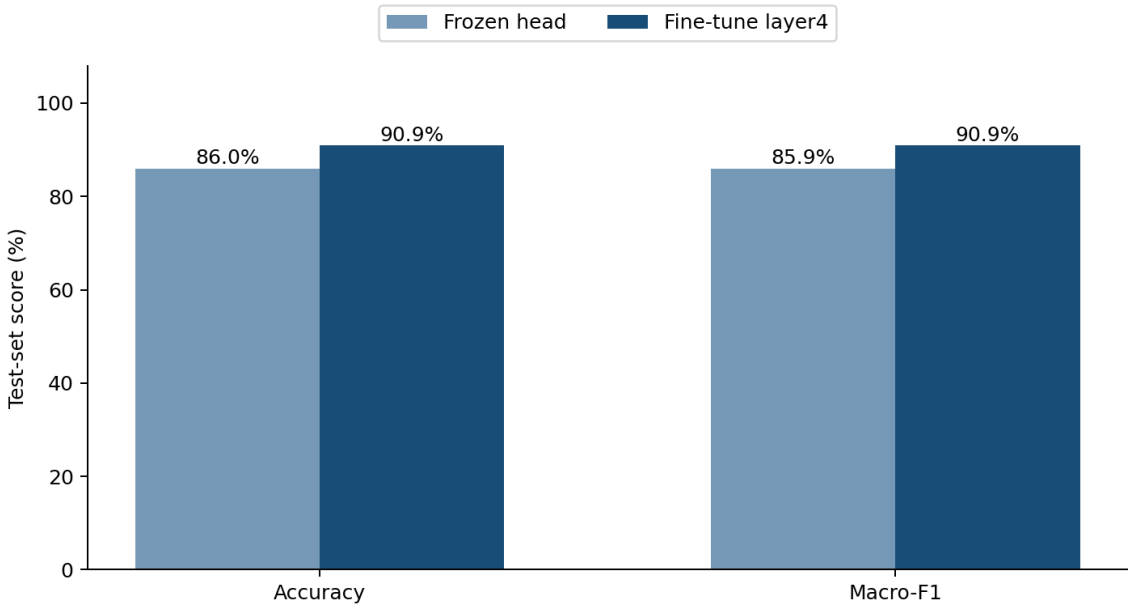


Figure 5. Held-out test accuracy and macro-F1 measured on the same 450 selected images.

Statistic	Frozen head	Fine-tuned layer4
Correct / tested	387 / 450	409 / 450
Accuracy	86.00%	90.89%
Macro-F1	0.8590	0.9090
Accuracy 95% bootstrap CI	82.67%-89.11%	88.22%-93.56%
Validation macro-F1 (selected)	0.8398	0.8888

The fine-tuned model improved accuracy by **4.89 percentage points** and macro-F1 by **0.0501** on the fixed held-out subset. Its 409 correct predictions represent 22 additional correct labels relative to the frozen head. The test error count fell from 63 to 41, a 34.9% relative reduction in errors on these images.

This performance exceeds both the non-learned 33.33% reference and the trained frozen-feature benchmark. Because the test set is balanced across three classes, accuracy is interpretable, but the macro-F1 increase confirms that the gain is not solely a class-prevalence effect.

10. Classwise results and confusion patterns

Class	Frozen precision / recall / F1	Fine-tuned precision / recall / F1
cat	0.799 / 0.927 / 0.858	0.899 / 0.893 / 0.896
dog	0.890 / 0.753 / 0.816	0.876 / 0.893 / 0.884
horse	0.906 / 0.900 / 0.903	0.953 / 0.940 / 0.946

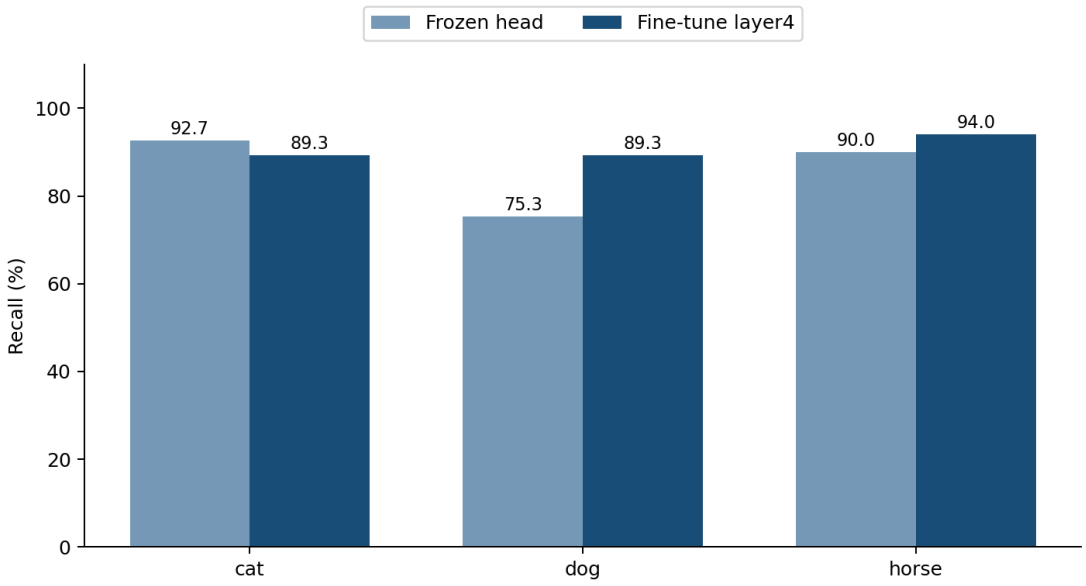


Figure 6. Test recall by true class. Fine-tuning improves dog and horse recall, while cat recall falls.

Observed errors. Baseline dog recall was 113/150 (75.33%), versus 134/150 (89.33%) after fine-tuning. Dog-to-cat errors decreased from 27 to 13; dog-to-horse errors from 10 to 3. Horse recall rose from 135/150 to 141/150. Conversely, cat recall fell from 139/150 to 134/150 and cat-to-dog mistakes increased from 7 to 12. This class-specific tradeoff is hidden by overall accuracy alone.

RESULTS

11. Confusion matrices and matched error analysis

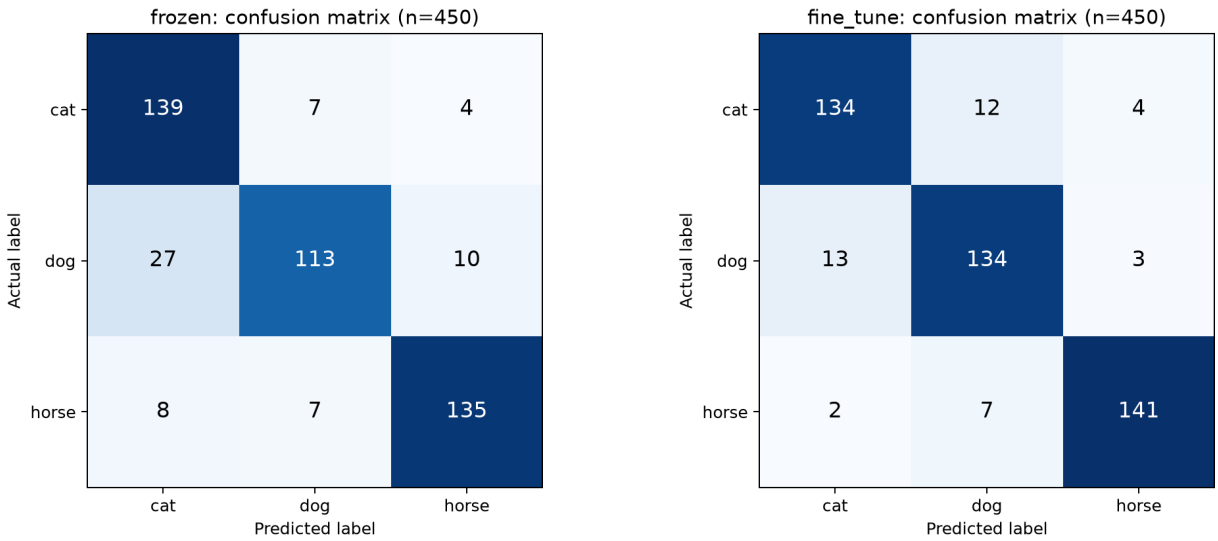


Figure 7. Frozen-head (left) and fine-tuned (right) test confusion matrices. Rows are true labels; columns are predictions.

Same test image	Count	Interpretation
Both models correct	372	Consistent correct predictions
Baseline wrong; fine-tune correct	37	Previously incorrect cases repaired
Baseline correct; fine-tune wrong	15	New mistakes introduced
Both models wrong	26	Shared failures

Matched predictions show 37 repaired mistakes but 15 new mistakes, for a net gain of 22. Most repairs occur in the dog class (24 corrected, 3 newly incorrect), while cat predictions exhibit the opposite pattern (5 corrected, 10 newly incorrect). This is why evaluating the models on exactly the same images is more informative than comparing independent summary scores.

12. Statistical validation, robustness and interpretation

Paired bootstrap. Let each held-out image contribute +1 if only the fine-tuned model is correct, -1 if only the frozen head is correct, or 0 if the two agree in correctness. The mean of these matched differences is 22/450 = **+0.0489**. We draw 2,000 bootstrap resamples of 450 matched image positions (with replacement; seed 42) and compute the 2.5th and 97.5th percentiles of the resampled mean difference.

Evidence	Estimate
Observed paired accuracy gain	+4.89 percentage points
95% paired bootstrap interval	+1.78 to +8.00 percentage points
Discordant outcomes	37 improved, 15 worsened
Exact two-sided McNemar test	p = 0.0032

Because the paired bootstrap interval excludes zero, the observed accuracy gain is supported by this image-level resampling analysis. An exact McNemar test on the 52 discordant image pairs also yields p = 0.0032, consistent with different matched error rates. These inferences are conditional on this fixed sampling design, fixed model instances and the assumption that held-out images are representative of the intended target population.

Validation and robustness boundaries

The dataset has only 450 held-out examples and one fixed random subset; three epochs and one training seed per strategy do not measure training-run variability. Individual confidence intervals overlap slightly and should not be used in place of the paired interval. The task is restricted to three CIFAR-10 labels and to heavily upscaled 32 x 32 pixels; no external photos, fairness groups, shifted imaging conditions or model calibration measurements were evaluated.

Moreover, the strategies use different learning rates and trainable parameter counts. Accordingly, the improvement supports the **combined fine-tuning configuration**, not an isolated causal claim about changing only one layer. The test images were used for final reporting and error analysis, not for choosing the saved checkpoints.

13. Conclusion, improvements and references

Conclusion. This capstone constructed a reproducible three-class animal classifier and compared two transfer-learning strategies. Partial fine-tuning achieved 90.89% test accuracy and 0.9090 macro-F1 versus 86.00% and 0.8590 for the frozen-feature baseline. The matched test accuracy gain was +4.89 percentage points, with a paired bootstrap 95% interval from +1.78 to +8.00 points. Dog recognition improved substantially; cat recall declined slightly, emphasizing the value of classwise inspection.

The tested configuration meets the defined objective on the chosen CIFAR-10 subset, but it is not proof of deployment readiness. The low-resolution, balanced dataset, single split/seed, three-epoch schedule and different optimizer learning rates limit generality and causal interpretation. No real-world robustness claim is made.

Next steps

A stronger follow-up would compare matched learning-rate grids using validation data, repeat each strategy across several independent seeds, add early stopping, inspect individual high-confidence mistakes, evaluate probability calibration and test a separately curated real-world animal-image dataset. None of these additional experiments is presented as completed work.

Reproducibility and references

The accompanying files include `capstone.py`, `analyze_results.py`, `make_report.py`, `requirements.txt`, tests, a saved split manifest, both training histories, both test-prediction files, metrics JSON, charts and a setup README. Model checkpoints and raw CIFAR-10 files are intentionally not included; the README explains how to obtain data and rerun training.

[1] Krizhevsky, A. (2009). *Learning Multiple Layers of Features from Tiny Images.* https://www.cs.toronto.edu/~kriz/cifar.html

[2] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition.* CVPR, pp. 770-778. https://arxiv.org/abs/1512.03385

[3] PyTorch. *Transfer Learning for Computer Vision Tutorial.* https://docs.pytorch.org/tutorials/beginner/transfer_learning_tutorial

[4] Torchvision. *ResNet18 weights and model documentation.* https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html

[5] Torchvision. *CIFAR10 dataset API.* https://docs.pytorch.org/vision/main/generated/torchvision.datasets.CIFAR10.html